

Zestaw powtórkowy – Kolokwium (wersja tablicowa)

Zadanie 1. Konwersje systemów liczbowych

1A. Konwersja z dziesiętnego na system pozycyjny (pełna, z obliczeniami)

Wykonaj konwersje liczb zapisanych w systemie dziesiętnym:

1. $156_{10} \rightarrow$ system dwójkowy
2. $0.625_{10} \rightarrow$ system ósemkowy
3. $93_{10} \rightarrow$ system szesnastkowy

W każdym przypadku pokaż kolejne kroki obliczeń (dzielenie przez podstawę, tabela reszt). Możesz zrobić przejście do wynikowej bazy pośrednio przez system dwójkowy np $10 \rightarrow 2 \rightarrow 16$. Będzie jeden z analogicznych wariantów (całkowita lub ułamek), losowo.

1B. Konwersja do systemu dziesiętnego (liczba całkowita ORAZ ułamek)

Przekonwertuj do systemu dziesiętnego.

Możesz zostawić wynik w postaci sumy potęg (wielomianu).

1. $3A7_{16}$
 2. 101.011_8
-

1C. Ułamek nieskończony $\rightarrow$ system dziesiętny

Przekonwertuj ułamek okresowy na zapis dziesiętny.

Wynik możesz zostawić w postaci ułamka p/q , p i q pozostawione jako wyrażenia arytmetyczne w systemie dziesiętnym.

$$0.(0101)_2$$

Zadanie 2. Algorytm iteracyjny

(pseudokod + schemat blokowy)

Napisz algorytm iteracyjny, który dla danej liczby naturalnej $n > 0$:

znajduje najmniejsze k takie, że

$$1^2 + 2^2 + \dots + k^2 \geq n$$

Wejście: $n > 0$, naturalne. Wyjście: $k > 0$, naturalne.

Wymagania:

- podaj pseudokod
- narysuj schemat blokowy
- użyj pętli iteracyjnej
- nie wolno używać wzoru zamkniętego

Zadanie 3. Rekurencja liniowa – implementacja wg opisu

Dana jest liczba naturalna $n \geq 0$.

Zdefiniuj rekurencyjnie funkcję, która oblicza:

Liczbę liczb cyfry n w zapisie dziesiętnym.

Wejście: n , naturalne. Wyjście: liczba cyfr n w systemie dziesiętnym.

Przykład:

dla $n = 5028$ wynik to 4

Warto rozważyć reszty z dzielenia przez 10 lub samo dzielenie.

Wymagania:

- podaj pseudokod
- jasno określ:

- przypadek bazowy
- wywołanie rekurencyjne

Zadanie 4. Śledzenie algorytmu (będzie JEDEN z wariantów, losowo)

4A. Śledzenie iteracyjne – tabela

Dany jest algorytm:

```
i := 1
s := 0
WHILE i ≤ 3 DO
  j := 1
  WHILE j ≤ 2 DO
    s := s + j*j
    j := j + 1
  END WHILE
  i := i + 1
END WHILE
```

Algorytm wykorzystuje dwie zagnieżdżone pętle `WHILE`.

Polecenie

1. Uzupełnij tabelę śledzenia wykonania algorytmu.
2. W tabeli należy uwzględnić:
 - stan początkowy (przed wejściem do pętli),
 - pierwszą iterację pętli zewnętrznej,
 - pierwszą iterację pętli wewnętrznej,
 - kolejne zmiany wartości zmiennych `i`, `j` oraz `s`.

Tabela śledzenia (początek)

krok	i	j	s	uwagi
start	1	–	0	przed pętlami
1	1	1	1	po wykonaniu <code>s := s + i*j</code>
2	1	2		
3	2	1		inkrementacja <code>i</code>
4	2	2		
...				

Pytania (jedno z nich, losowo)

- Ile razy wykona się instrukcja `s := s + j*j` ?
- W którym momencie następuje wyjście z pętli wewnętrznej?

4B. Rekurencja liniowa – call stack

Dana jest funkcja:

```
f(n):
  IF n == 0
    RETURN 0
  RETURN n + f(n-1)
```

Wypisz stos wywołań (call stack) dla $f(4)$
(w kolejności rzeczywistego wykonania).

4C. Rekurencja rozgałęziona – drzewo wywołań

Dana jest funkcja:

```
g(n):
  IF n <= 1
    RETURN 1
  RETURN g(n-1) + 2*g(n-2)
```

Narysuj drzewo wywołań dla $g(4)$ oraz pokaż logiczno formalny porządek wykonania obliczeń, stopniowo zastępując liście obliczonymi wartościami funkcji.

Zadanie 5. Pamięć i zakres liczb

5A. Zakres liczb

1. Ile ujemnych liczb całkowitych można przechować na 256 bitach, jeśli 1 bit przeznaczony jest na znak?
 2. Podaj zakres wartości min i max.
 3. co w wypadku, gdy przechowujemy tylko liczby całkowite nieujemne?
-

5B. Rozmiar tablicy danych

Założmy, że przechowujemy liczby z punktu 5A w tablicy:

$A^{1120,8}$

1. Ile bitów zajmuje cała tablica?
2. Ile to bajtów?

(Wynik można zostawić w postaci wyrażenia arytmetycznego.)